MP Chatbot: An NLP-based Flight Booking Chatbot

MATTHEW POON*, COMP3074, Human-AI Interaction, University of Nottingham, UK

This report presents a conversational chatbot that supports flight booking, small talk, and question answering. It combines a machine learning intent classifier with a template based dialogue flow. The report expands on the system architecture, key design choices for conversational flow, and evaluates the chatbot through model performance and usability testing.

Additional Key Words and Phrases: Chatbot, Conversational AI, Machine Learning, Intent classification, Usability, Flight booking, Small talk, Discoverability, Transactions

ACM Reference Format:
Matthew Poon. 2025. MP Chatbot: An NLP-based Flight Booking Chatbot. 1, 1 (December 2025), 15 pages.

1 Introduction

MP Chatbot is a conversational system interacted with via the command line, able to support flight booking, small talk, and question answering. The system follows a hybrid architecture; a machine learning intent classification model identifies user intent, while a rule-based dialogue flow handles the booking sequence and controls how the chatbot navigates between different steps. When the classifier cannot assign an intent with enough certainty, it falls back to a TF-IDF system that queries a Question/Answer database to retrieve a suitable reply.

2 Chatbot Architecture

2.1 System Overview

The system is implemented using a modular architecture, where each major function is separated into its own component. A high level system architecture diagram is shown in Figure 1. It demonstrates the command line interface receiving input from the user, which is passed into the dialogue manager. No preprocessing occurs at this point, until the message is routed to the machine learning intent classifier, which uses TF-IDF and the model pipeline to predict one of several intents. If the predicted intent is of low confidence, the system routes the message to the fallback Q/A system instead.

Based on the determined intent, the dialogue manager chooses the next corresponding system action by accessing the shared *user_state* object, storing all context (user's name, booking progress, selected flights, follow up prompts). Once a specific action is invoked, different results occur: the small talk action produces template driven responses, the Q/A action retrieves answers from a TF-IDF similarity search on a dataset, and the flight booking action runs the transactional flow.

With this architecture, all the logic from different components are kept separate, allowing for easier code interpretability, and ease of additions in future. A modular design isolates state handling, intent detection, and transactional logic, to prevent errors carrying over into multiple components.

Author's Contact Information: Matthew Poon, psymp18@nottingham.ac.uk, COMP3074, Human-AI Interaction, University of Nottingham, Nottingham, UK.

2025. ACM XXXX-XXXX/2025/12-ART
https://doi.org/

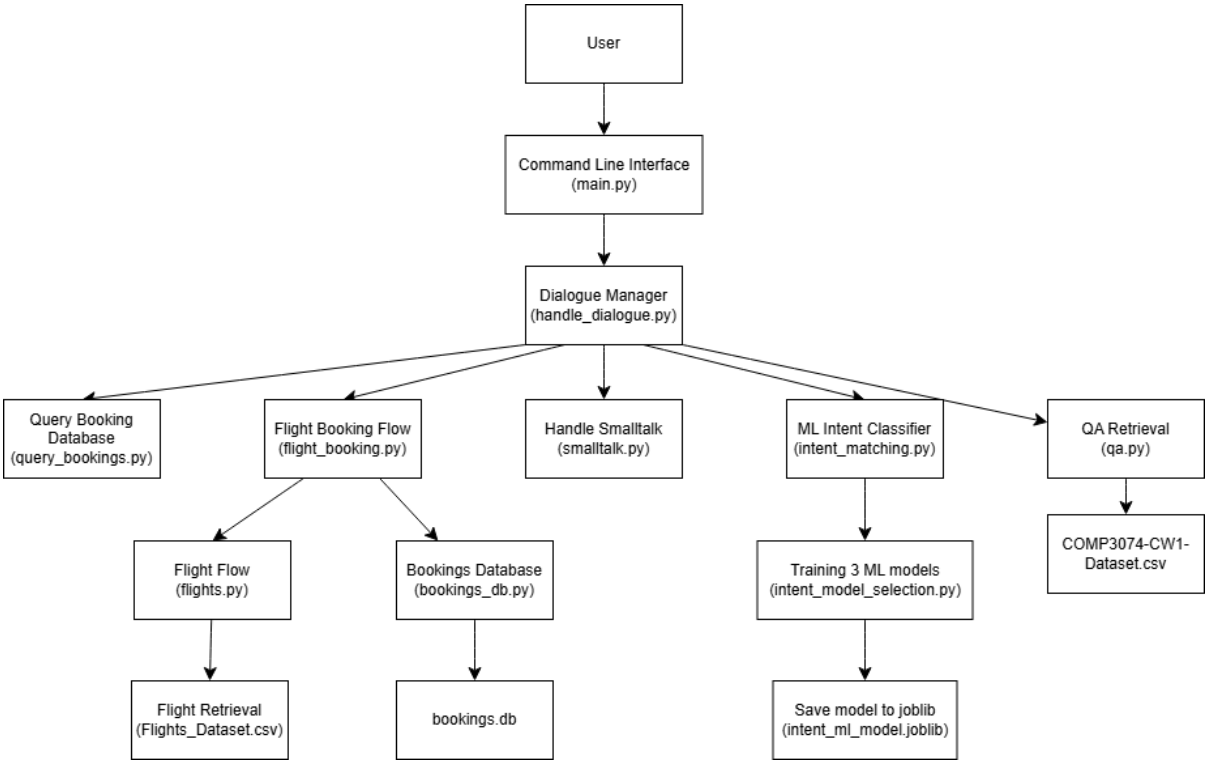


Fig. 1. System Architecture Diagram

2.2 Software Components and Data Flow

Taking a closer look at the specific software components, we can analyse the system architecture further and describe how these different components interact with each other. The system is built from a set of modular files, where each file has a single specific purpose, keeping code easy to follow and clearly structures the flow of data.

All input enters the system through *main.py*, which sends each message to the dialogue manager, *handle_dialogue.py*, acting as the central controller of the entire system. By tokenising the raw text and calling the intent classifier when needed, messages are routed to the correct module based on the classified intent.

The intent classification pipeline consists of loading the classifier and predicting the most likely intent. The training script, *intent_model_selection.py*, is separate and loads the dataset, runs 10-fold cross-validation across three algorithms, and chooses the best model based on their accuracy.

If the predicted intent score falls below the threshold, the system switches to the fallback Q/A module, which retrieves answers based on TF-IDF similarity scores. A fallback retrieval system was chosen here as the classifier struggles with open-ended user queries, so through using TF-IDF, the system has a more robust similarity matching system with low computational cost for these specific queries.

Small talk is handled by *smalltalk.py*, containing a set of template responses. An aggregation implementation selects a single response from a set of candidate responses that match the query. Template-based responses were used as they were a simple way of guaranteeing grammatical correctness and avoid unwanted phrasing of responses.

Transactional flow is handled by the flight booking module, *flight_booking.py*, which uses the shared user state to keep track of slot filling (origin, destination, dates, and trip type). It represents a structured transactional flow, where the system moves through a sequence of defined stages to reach the booking goal. Based on the provided booking details, matching entries are listed, allowing the user to confirm a booking and store it in an SQLite database which they can query.

All system actions return some text to the dialogue manager, which is then outputted to the user on the command line. The separated pipeline ensures dialogue logic is not mixed within the classifier, so each system layer is completely independent from the rest.

2.3 NLP Pipeline for Intent Detection

This section will further explore the Natural Language Processing pipeline, which follows a machine learning approach by training and selecting a classifier to detect user intent, as shown in Figure 2. The system is trained on *Intent_Training_Dataset.csv*, containing labelled examples of user inputs mapped to corresponding intent categories, and converts the text into numerical form using a TF-IDF Vectoriser. This was chosen over more complex embeddings as the dataset is small, and TF-IDF is more than capable of providing a suitable performance at minimal cost.

Three candidate models are tested: Logistic Regression, Linear SVM, and Multinomial Naive Bayes, all of which are wrapped into a pipeline combining the vectoriser and classifier. The training script then performs 10-fold cross validation for each model, evaluating the accuracy, precision, recall and F1 score. From this, the model with the highest F1 score is selected to be retrained using the full dataset and exported as a *.joblib* file for runtime usage. Figure 2 shows the separation between training and runtime execution, ensuring user interactions never trigger training code, keeping runtime small.



Fig. 2. Machine Learning Pipeline Flowchart Diagram

During runtime, the system loads the trained model and applies it to each incoming message. The TF-IDF vectoriser in the saved pipeline transforms the text into numerical form, and the classifier then predicts the intent label. An important part of the runtime pipeline is displayed in Figure 3, the probability-based confidence check. The threshold acts as a control mechanism to prevent the classifier from making unwanted predictions. When uncertainty is high (below the threshold) the fallback Q/A module is called upon to provide a safe response.

```
5   #determine intent using ML model
6   def detect_intent(user_query: str):
7       pred = pipeline.predict([user_query])[0]
8       proba = pipeline.predict_proba([user_query])[0].max()
9
10
11      if proba < 0.35: #threshold
12          return "fallback", proba
13
14      return pred, proba
```

Fig. 3. Probability Confidence Check Code Snippet

2.4 Question Answering and Fallback Retrieval

This fallback module uses a TF-IDF similarity search to retrieve answers from the Q/A dataset, where each question is vectorised at startup. When the user asks a question that can be answered by the dataset, the system computes its vector representation and measures cosine similarity between it and the stored vectors. If the best similarity score exceeds the threshold, the corresponding answer is returned to the user. Otherwise, the system will fallback and ask them to rephrase their question. This gives the chatbot a wider range of responses that expand the scope of just the flight booking transaction, without the need of the intent classifier.

2.5 Flight Booking Transaction Flow

The flight booking module is the handler for the main transactional flow of the chatbot. It processes user input through a determined sequence of states, as well as slot filling and route validation. This was used as the transaction requires specific flight information, so by forcing the user into a fixed sequence, the system is able to gather all the relevant details without errors. Figure 4 illustrates the complete transactional flow, starting from the moment the user triggers the booking intent. This is implemented in *flight_booking.py*, and represents the most detailed part of the system architecture.

The transaction begins when the intent classifier detects the *start_flight_booking* intent. The system prompts the user for the first missing slot, the origin city. It then iterates through a fixed slot order and checks if the input is valid at each slot.

Each user input is handled by *handle_booking()*, which performs each corresponding action based on the slot. For example, for input that represents cities, it simply stores the raw text. However, for date fields the system uses the date normalisation utility function *normalise_date()* to convert natural language formatted dates into a consistent DD/MM/YYYY format the system can understand.

It checks whether the origin and destination inputted by the user form a valid route in the flight dataset, *Flights_Dataset.csv*. If it does not exist, both fields are reset and the user is prompted to choose a new route, to prevent the booking from progressing further when the system already knows there are no available flights. Once all required slots in the sequence are filled, the system moves to the completion stage, where it returns a filtered list of matching flights. If no matching flights are found, the date field resets and the user is prompted to change their date as an error handling mechanism.

The flow is split up between single flights and return journeys to make logic easier to track. For single flights, the chatbot will perform simple aggregation through generating a set of potential descriptions, and list all matching flights. Once listed, the user must input a valid flight ID number, which is extracted and checked against the list to see if it matches an entry. Once the user has confirmed their flight, the system stores their booking in a database and resets all booking states.

Similarly, the return journey works in the same manner, but involves two linked searches instead. After the user has selected a valid outbound flight, the system switches state to expect a return ID. The final stage computes the combined price of both flights and returns a confirmation message to the user.

This flight booking system ensures that there is a set sequence of operations that must be followed, removing ambiguity and hanging interactions. This allows for more robust transactional behaviour and introduces clear methods of error handling.

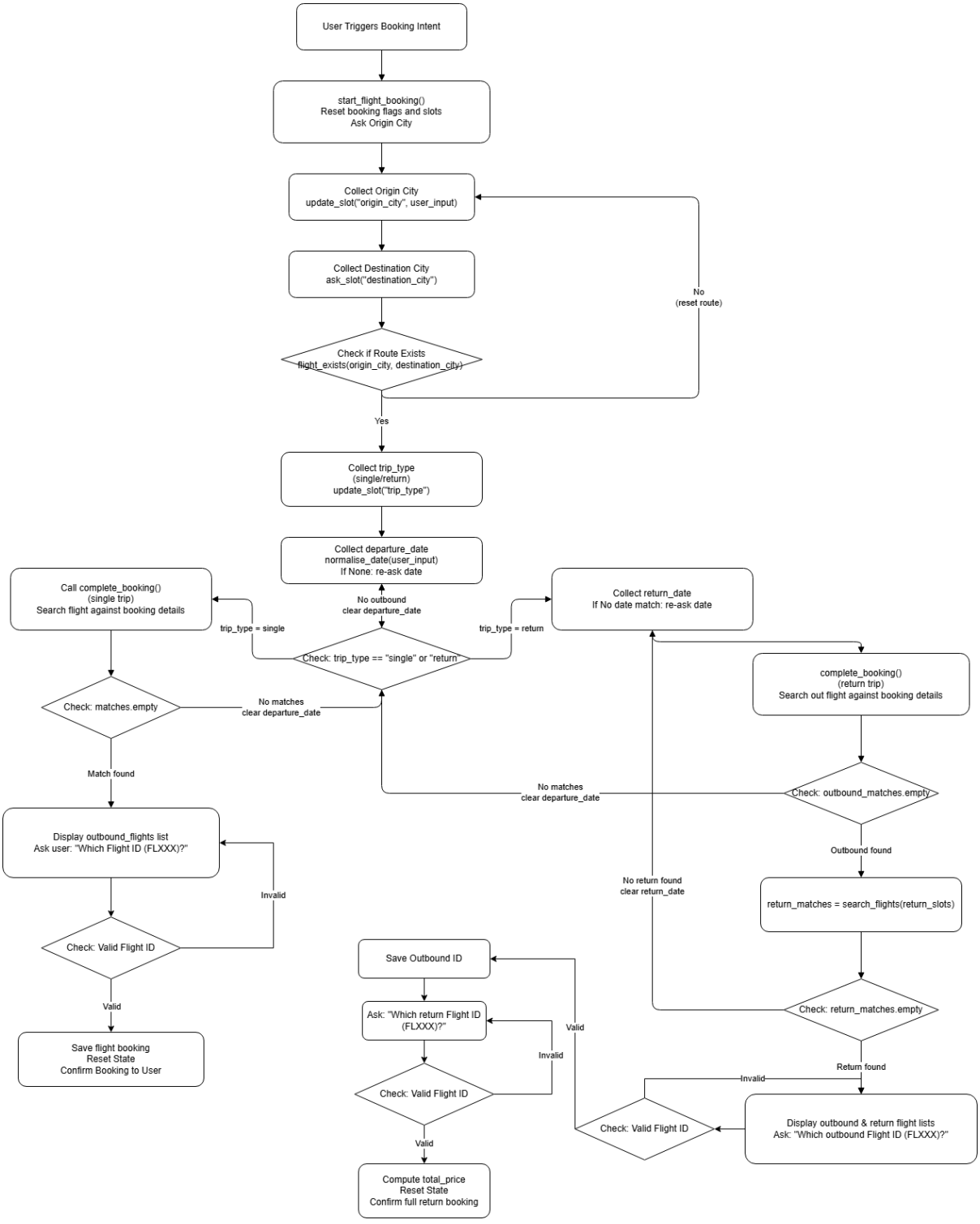


Fig. 4. Flight Booking Flowchart

2.6 Dialogue State and Persistence

The system relies on a shared state object, *user_state*, to keep track of user information throughout every conversation. The states are displayed below in Figure 5.

```
user_state = {
    "name": None,
    "greeted": False,
    "booking_active": False,
    "pending_single_flight": False,
    "pending_outbound_flight": False,
    "pending_return_flight": False,
    "selected_outbound_id": None,
    "pending_followup": False,
    "chat_id": None,
    "booking_slots": {
        "origin_city": None,
        "destination_city": None,
        "departure_date": None,
        "trip_type": None,
        "return_date": None,
    },
}
```

Fig. 5. States Code Snippet

3 Conversational Design

The chatbot aims to implement various conversational design principles, in order to produce clear and predictable interactions that support both open-ended dialogue and sequential transactional flow. How the system interprets input, maintains context, and produces relevant responses are all shaped by these design principles.

3.1 Prompt Design

The system uses clear, short, and polite prompts that guide the user through the booking transaction. Conversational markers such as "Okay", "Great", and "Got it", are used to help maintain flow whilst keeping the chatbot sound less robotic. Simple aggregation is used for small talk as shown in Figure 6, to add depth to responses ensuring that the chatbot avoids sounding too repetitive.

```
def closing():
    return choose(["Okay. To end this chat, please type 'stop'.",
                   "Alright. If you'd like to finish, just type 'stop'.",
                   "Got it. When you're ready to end the chat, type 'stop'."])

```

Fig. 6. Simple Aggregation Code Snippet

During flight booking, each prompt corresponds to a single required slot, seen in Figure 7. This ensures content determination is explicit, as the system decides only the next missing slot should be prompted. Prompts are short and to the point, helping users stay oriented within the transaction.

```
35  def ask_slot(slot):
36      if slot == "origin_city":
37          return "Which city are you flying from?"
38      if slot == "destination_city":
39          return "Thanks. And which city are you flying to?"
40      if slot == "trip_type":
41          return "Now, is this a single (one way) flight or a return flight?"
42      if slot == "departure_date":
43          return "And on what date would you like to depart?"
44      if slot == "return_date":
45          return "Finally, on what date would you like to return?"
46      return "Could you provide more details for your flight?"
47
```

Fig. 7. Slot Prompting Code Snippet

3.2 Discoverability

The principle of discoverability is implemented through the system's aggregated responses, including the *discoverability()* and *thanks()* functions shown in Figure 8 below. They are conversational guides that assist the user in discovering the functionality of the chatbot and aids follow up responses. Together, these ensure that the user always understands what the chatbot is able to do, and what next steps need to be taken. Figure 7 shows that discoverability cues are essential in helping users understand what actions are possible at each step.

```
10  def discoverability():
11      return choose(["I can help you with booking a flight. I can also answer any question you may have, too.",
12                     "I can book flights for you and answer any general questions you may have.",
13                     "I'm here to help you book flights and to answer questions you may have."])
14
24  def thanks():
25      return choose([f"You're welcome {user_state['name']}! Is there anything else I can help you with?",
26                     f"No problem, {user_state['name']}. Would you like help with anything else?",
27                     f"No worries {user_state['name']}. Is there anything else you need help with?"])
28
```

Fig. 8. Discoverability Functions Code Snippet

3.3 Error Handling

Throughout the whole system, robust error handling techniques have been implemented to ensure interactions are not halted due to errors, but redirected to guide the user back to a valid conversational flow. Some examples of this include flight date normalisation error handling and validating flight IDs, as outlined in Figure 9. Error handling methods were designed to explain what went wrong with input and informs the user of the next required action to amend the error.

```
78      elif slot in ("departure_date", "return_date"):
79          norm = normalise_date(input)
80          if norm is None:
81              return "Sorry, I don't understand that date. Try 'DD/MM/YYYY'."
82          update_slot(slot, norm)
```

```
210     if match.empty:
211         return "I couldn't find that flight ID in the list above. Please type one of the IDs from the list (FLXXX)."
```



```
MP:  And on what date would you like to depart?
You: 200 december
MP:  Sorry, I don't understand that date. Try 'DD/MM/YYYY'.
You: 20/12/2025
MP:  There are 2 flights from London to Paris on 20/12/2025:

- FL001: London to Paris on 20/12/2025 at 08:10 (Economy, £89)
- FL002: London to Paris on 20/12/2025 at 15:45 (Economy, £105)


Which flight ID would you like to book? (FLXXX).
You: fl003
MP:  I couldn't find that flight ID in the list above. Please type one of the IDs from the list (FLXXX).
```

Fig. 9. Error Handling Example 1

Additionally, when the user enters an invalid route path, the system handles the error by telling the user why why it was wrong and what they must do to correct it, as per Figure 10. As shown, it provides reasoning and a next action to the user to prevent any confusion.

```
86      if slots.get("origin_city") and slots.get("destination_city") and slot in ("origin_city", "destination_city"):
87          if not flight_exists(slots["origin_city"], slots["destination_city"]):
88              false_origin = slots["origin_city"]
89              false_dest = slots["destination_city"]
90
91              slots["origin_city"] = None
92              slots["destination_city"] = None
93
94              return f"Sorry, I couldn't find any flights from {false_origin} to {false_dest}. Please choose a different origin city."
95
```



```
MP:  Great! I can help with that. Firstly, where are you flying from?
You: London
MP:  Thanks. And which city are you flying to?
You: Brussels
MP:  Sorry, I couldn't find any flights from London to Brussels. Please choose a different origin city.
```

Fig. 10. Error Handling Example 2

Moreover, the intent classification implementation is protected through error handling, by explicitly falling back to Q/A when predicted intent confidence is too low. If the classifier is uncertain, the system will avoid predicting the wrong intent and instead provide a safe alternative, as shown previously in Figure 3.

Robust error handling was essential as transactions carry higher impact if mistakes are not dealt with. Providing explanations and guidance ensures errors become recoverable rather than failures.

3.4 Personalisation

The shared *user_state* object ensures the system is personalised to each user. Figure 11 shows the system storing the user's name after greeting and explicitly mentions it in many responses throughout the chat flow. Through stored

user details, user engagement is strengthened and repetition is reduced, supporting a more natural conversational experience.



Fig. 11. Personalisation Example

3.5 Confirmation Strategies

The chatbot uses both implicit and explicit confirmation strategies to ensure the booking flow remains stable. Explicit confirmation can be seen above in Figure 9 when the system requires a valid flight ID before confirming a booking. After booking is completed, the user is able to confirm details by asking the bot and upon request, the chatbot retrieves the record and returns a summary to the user, as seen below in Figure 12.

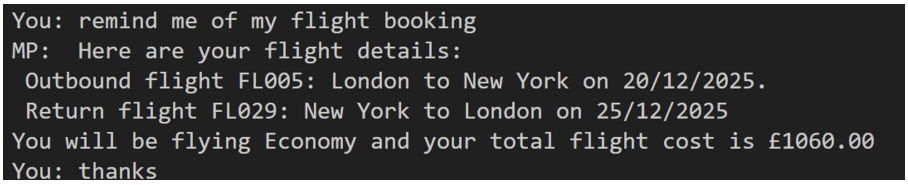


Fig. 12. Explicit Flight Confirmation

Additionally, implicit confirmation appears through clarifying questions. For example, Figure 13 demonstrates the chatbot checking whether the user has entered a trip type (single or return) by asking a clarifying question. These strategies ensure that instructions, choices, and transitions leave no room for confusion. Using both explicit and implicit confirmation ensures essential booking details are verified before progressing, reducing the risk of incorrect transactions, or acting on incomplete information.

```
70      if slot == "trip_type":
71          lower = input.lower()
72          if "return" in lower or "round" in lower:
73              update_slot("trip_type", "return")
74          elif "single" in lower or "one way" in lower:
75              update_slot("trip_type", "single")
76          else:
77              return "Is that a single (one way) or a return flight?"
```

Fig. 13. Flight Type Clarification

3.6 Context Tracking

Context tracking is handled through the shared *user_state* object, which maintains information across every conversation turn. From this, the chatbot can maintain flow without needing to re-ask for details.

A key part of context tracking is how the chatbot switches between general conversation and the booking transaction, by using a gate shown in Figure 14. Based on this, incoming messages are routed to the booking handler to prevent small talk or Q/A retrieval interrupting the transaction. This design ensures transactional flow is maintained and prevents irrelevant intents from interrupting the booking sequence

```
47      if user_state["booking_active"]:
48          return handle_booking(user_input)
```

Fig. 14. State Flag Code Snippet

Within the actual booking flow, flags are used to maintain further context. They represent sub-states of the transaction to ensure each step is correctly handled by it's corresponding function. This means the system always understands what point in the transaction it is in, and prevents the flow from losing track of the user's progress, supported by Figure 15.

```
54      if user_state["pending_single_flight"]:
55          return handle_single_flight(input)
56      if user_state["pending_outbound_flight"]:
57          return handle_outbound_flight(input)
58      if user_state["pending_return_flight"]:
59          return handle_return_flight(input)
```

Fig. 15. Pending Flags Code Snippet

4 Evaluation

The chatbot was evaluated using two methods: performance testing for the intent classifier, and usability testing with real users interacting with the system. Together, these evaluation metrics assess both how well the system recognises user input, and how the chatbot works in practice.

4.1 Performance Evaluation for Intent Classification

Machine learning model performance was measured through 10-fold cross-validation on the intent dataset. Figure 16 compares Logistic Regression, Linear SVM, and Multinomial Naive Bayes, where all three models achieved high scores, but Logistic Regression achieved the highest F1 mean (0.966).

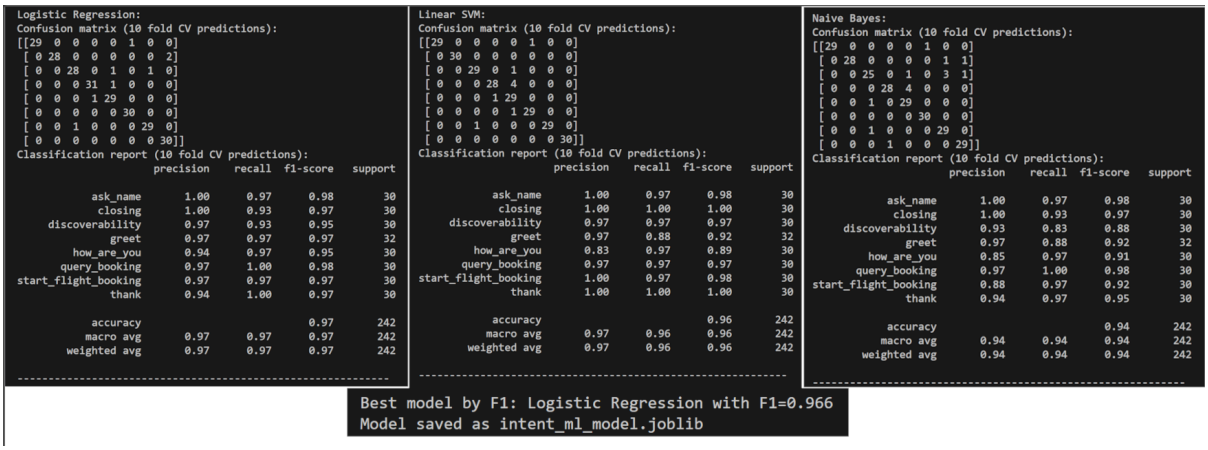


Fig. 16. Performance Metric Results

4.2 Usability Evaluation

Four participants completed a Likert scale Chatbot Usability Questionnaire, with results visualised using a heatmap displayed in Figure 17. Participants were given a list of tasks to complete using the chatbot, such as finding out what the chatbot can do, and booking a flight. The questionnaire then assessed clarity of chatbot responses, ease of booking, error handling, and overall satisfaction.

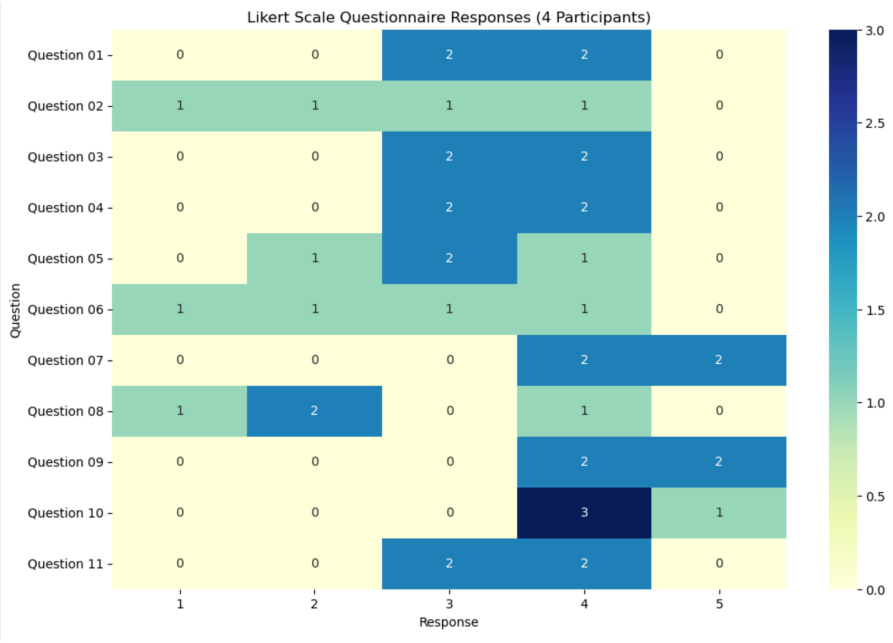


Fig. 17. Usability Heatmap

5 Discussion

5.1 Interpretation of Performance Results Given RRI

Looking at the results of the performance metrics, all three classifiers produced high accuracy, precision, recall and F1 scores, with Logistic Regression performing best overall. The confusion matrices in Figure 16 show highly accurate classification results across all eight intent categories. Most misclassifications occurred in the *discoverability* and *how_are_you* intents, due to having many overlapping words. Other intents such as *thank* and *ask_name* had values of 0.97-1.00, demonstrating strong results. However, high values close to 1.00 may indicate that the classifier is learning the training data phrasing, rather than general patterns.

Given a Responsible Research and Innovation (RRI) perspective, these results may imply that because the dataset is small and manually curated, high results may not reflect real-world use. A responsible approach identifies that future development should include inclusive, real-world data collection to cater to a more diverse range of expressions users may say, in order to reduce bias towards particular linguistic styles. Expanding the dataset this way will ensure a more socially responsible approach that is less prone to exposing biases in training data.

5.2 Interpretation of Usability Results Given RRI

As visualised in Figure 17, overall feedback from participants was positive; the system was consistently rated as easy to use, with most responses falling in the 4-5 range (agree and strongly agree). The stacked bar chart in Figure 18 supports this, displaying participants reported that the chatbot explained its purpose clearly and was welcoming throughout the interaction, based on the same 4-5 range frequency in answers. The flow of these interactions were deemed as simple to navigate, with the booking transaction being highly rated in terms of

clarity and ease of understanding. Error handling was also viewed positively, where participants felt the chatbot recovered well from certain mistakes.

Two areas did not perform as well: some participants stated the lexical choices used by the chatbot seemed robotic at times. Additionally, some mentioned some confusion during parts of the interaction, stating that the chatbot sometimes failed to understand them. With these results, overall it can be argued that participants found the system easy to use, were able to complete their assigned tasks, and overall were generally satisfied with the system.

However, with only 4 participants, this questionnaire may not capture the full range of different conversations users may have, and although the CUQ tasks ensured full assessment of the core features, it may not reflect real-world interactions in non-restrictive environments. Given an RRI view, a wider range of participants would make testing more robust; including users from diverse backgrounds and with varied literacy levels would provide a more accurate understanding of the system's strengths and weaknesses.

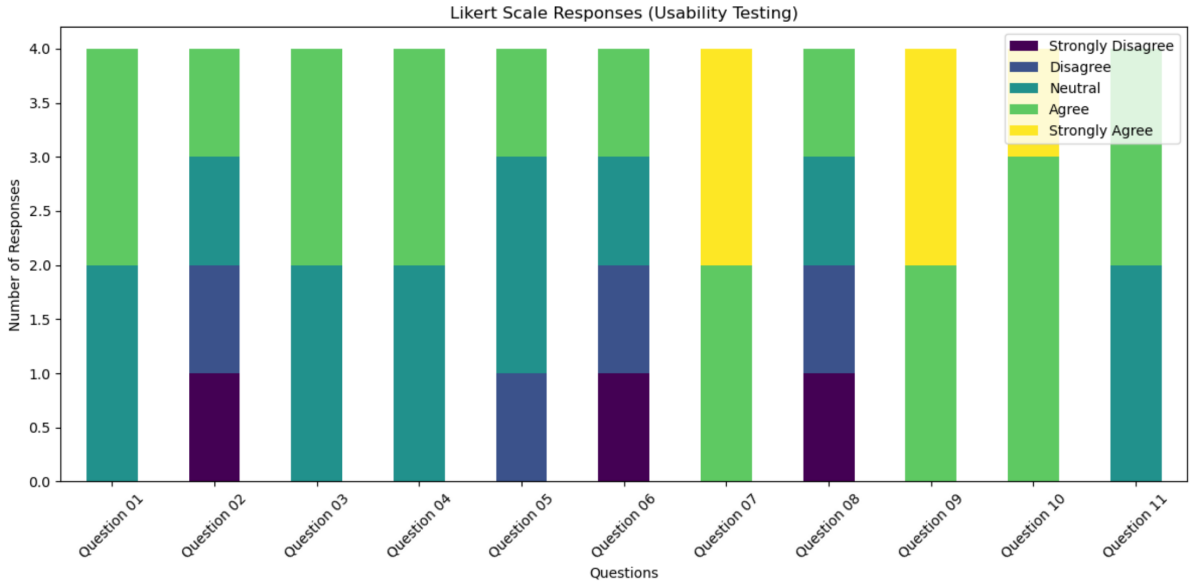


Fig. 18. Usability Stacked Bar Chart

5.3 Reflection on Application Area

The application area of this system is travel assistance, where aiding users with booking flights is a familiar task for many existing systems. From an RRI perspective, consideration of user expectations, potential risks, and societal impacts must be accounted for in such a system.

Users may place their trust in these travel chatbots, so the system must communicate precisely and clearly in order to prevent being misleading. The system must be robust enough to mitigate the risks, especially with incorrect dates or routes, as this may have negative consequences for users. The structured transactional flow helps to reduce these risks, but future implementations would benefit from stronger verification steps, such as confirming a chosen booking.

Societal impacts should also be considered for this application area, as while automated flight booking chatbots may increase efficiency, they may also replace human workers, or disfavour users who struggle with text-based interfaces.

Given these RRI principles, future iterations of the system should cater towards greater inclusivity and diversity of users. These reflections show that straightforward systems like these must still be developed with attention to the broader impact they may have.

6 Conclusion

This report has presented the design, implementation, justification, and evaluation of MP Chatbot, a conversational system used to book flights and answer general questions.

Evaluations of the system demonstrated its robustness, performing well when classifying intents and allowing users to complete tasks with minor confusion. These results demonstrate that the approach taken when designing and implementing this system has adhered to conversational design principles.

Reflection through the view of Responsible Research and Innovation highlighted the importance of accuracy, transparency, and inclusivity. Transactional chatbots like these carry risks if they do not communicate clearly or fail to guide the user correctly. Future versions aim to address these concerns, by expanding datasets and broadening error handling edge cases.

Overall, the system implements a modular architecture that is easy to interpret, all while providing a useful and user-friendly conversational system.